Lecture 1: Working at the Unix Command Line

Big Data Algorithms

> **Mac users**
>
> Almost everything here works the same in the Mac Terminal. The exceptions are small: the Linux commands `free` and `nproc` don't exist on a Mac (use `top`, and `sysctl -n hw.ncpu`), `ps aux --sort=...` and `stat -c` are Linux-only forms, and the shell's startup file is `~/.zshrc` instead of `~/.bashrc`.

Before we start Linux, let's make sure you have R and Python setup correctly. Here are two chunks that should test it. Go to console on the bottom pane and click the "+" sign to start both an R session. Then click the down icon next to plus and click "start another" and choose the python in

```
7 + 3
```

```
[1] 10
```

```
7 + 3
```

```
10
```

Assuming you're using Positron, we need the ability to send commands from this `qmd` document directly to terminal. (By default this is only enabled for R or Python). So follow these instructions:

Open File → Preferences → Keyboard Shortcuts, or press Ctrl+K and then Ctrl+S. In the search box, type "Run Selected Text In Active Terminal". Click the pencil icon next to that command, press the keys you want (for example Ctrl+Alt+Enter), then press Enter. You don't want to use Ctrl+Enter as this will conflict with R later.

We first create a subfolder for scratch work for this lecture and move there using `mkdir` (make [new] directory [if it doesn't exist]) The `~` is a useful shortcut; it means "home directory". W

```
mkdir -p ~/workspace/HUJI_52002_Fall_2026/scratch/lec01 # -p also creates the parent folders
```

and `cd` (change directory) into this new directory

```
cd ~/workspace/HUJI_52002_Fall_2026/scratch/lec01
pwd
```

The file system is a tree. You (via the linux prompt) are always "in" one directory, called the **working directory**. Let's verify we are in the right directory using `pwd` (print [the] working directory).

```
pwd
```

Download data

We will start with `covid.csv`, from the New York Times' public repository of COVID-19 data. The file tracks COVID-19 in the United States during 2020: each row gives one county on one day, with the total cases and deaths reported there up to that day. It has about 885,000 rows and takes about 35 MB. We won't use it until much later in this lecture except trivially.

Our first "real" linux command is `curl` (client URL) which downloads whatever is at the web address (a URL) you supply.

```
curl -L -o covid.csv https://raw.githubusercontent.com/nytimes/covid-19-data/master/us-counti
#^^^  ^^^^^^^^^^^^^^ ^^^^^^^^^
#  |  cmd options    cmd argument
# linux command
```

A command has three parts: the **command** itself, **options** (also called flags) that change its behavior, and **arguments** that tell it what to act on.

We gave it two options:

- `-L` follows redirects: if the server says the file has moved, `curl` automatically fetches it from the new location.
- `-o covid.csv` saves the download in a file with that name. Without `-o`, `curl` prints the contents to the screen.

You don't have to memorize options like these. Almost every command comes with a manual page that lists them, which you open with `man`:

```
man curl      # the full manual page (type /--output to search, q to quit)
curl --help   # a shorter summary (GNU/Linux tools)
```

Now let's make sure the `curl` actually worked i.e. downloaded the website and named it correctly. For this we'll use our next command `ls` (list [contents of a directory]). We will use a few options

```
ls
ls -l           # list with "l"ong details (default sort is by alphabetical order)
ls -l -h        # + list with "h"uman readable file sizes
ls -l -h -a     # + list "a"ll including hidden files
ls -lha         # `ls -lha` is the same as `ls -l -h -a` this is my go-to
ls -lhaS        # my other go-to as it also sorts by size (directory links are 4KB - interes
ls -lht         # newest file first (t = sort by time)
ls -lhtr        # oldest first (r reverses any of these)
```

And type of file is it (the filename can lie)?

```
file covid.csv
```

The . and .. are "hidden" and signify special directory entries known as "hard links" that act as shortcuts for file system navigation. The `.` is the current directory. We will use it soon. The `..` is the parent directory e.g.

```
cd .    #does nothing as it's changing directory into the current directory
pwd
cd ..   #visit one directory up
pwd
```

An **absolute path** starts from the root, `/` so the following will fail

```
cd /lec01
```

This fails because there is no `lec01` directory in the root:

```
ls /
```

The following is the listing of all files and directories from the root. If you're a sysadmin, you'll be using logs in the root folder `var` a lot and installing programs to `bin` and `opt`. But this is not a class in linux sysadmin - we only need to know what's useful for the data world.

A **relative path** starts from where you are i.e. beginning with pwd. So

```
cd lec01
```

is short for

```
cd /home/kapelner/workspace/HUJI_52002_Fall_2026/scratch/lec01
#  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
#  this is the current working directory
```

Text file stats via `wc` (word count)

```
wc -l covid.csv                # count the number of lines in the file
wc -lwmc covid.csv             # count the number of lines, words, characters, bytes in the
```

How long does a command take? Put `time` in front of it:

```
time wc -l covid.csv
```

`real` is the wall-clock time. We'll use `time` to compare methods as well as other profiling techniques throughout the semester.

Keyboard habits that save hours:

Key	What it does
Ctrl+L or `clear`	Clear the screen
Tab	Completes file and command names e.g. wc co Press twice to see options.
↑ / ↓	Scroll through previous commands
Ctrl+R	Search your command history (Enter to run, Ctrl+R again to get previous, Ctrl+G to clear)
Ctrl+C	Stop the running command
Ctrl+U	Clear the line

```
history             # list your recent commands
which python3       # where is this program installed?
echo $HOME          # print a variable: your home directory (what "~" means)
echo $PATH          # the directories searched for programs
echo $PATH | grep "bigdata" #we will see what this means later
```

The PATH variable is pretty important. You will be adding directories to it for convenience at times so you don't need to specify absolute path each time you want to run a new program you installed.

How much space are things taking?

```
du -sh .                                          # total size of this directory
du -sh ~/workspace/HUJI_52002_Fall_2026           # or of any directory of interest
du -h -d 1 ~/workspace/HUJI_52002_Fall_2026       # size of each subdirectory in repo root
df -h                    # free space on every disk and partition (note tmp goes in RAM - cl
df -h ~                  # free space on the disk holding your home directory (what you actu
```

Managing files

```
touch notes.txt             # create an empty file (or update its timestamp)
ls -alhS
cp notes.txt notes2.txt     # copy a file
ls -alhS
mkdir tmp_dir               # make a directory (we've seen this before)
ls -alhS
cp -r tmp_dir tmp_dir_copy  # copy a directory (-r = recursive)
ls -alhS
mv notes2.txt tmp_dir/      # move a file to another folder
ls -alhS
ls tmp_dir/ -alhS
mv tmp_dir/notes2.txt .     # move it to the working directory (that `.`!)
ls -alhS
mv notes.txt my_notes.txt   # rename (via moving to a new name)
ls -alhS
rm notes.txt                # delete a file
ls -alhS
rm notes2.txt my_notes.txt  # delete multiple files
rm -r tmp_dir               # delete a directory and everything in it (r is for "recursive" w
ls -alhS
rm tmp_dir_copy             # fails
rm -r tmp_dir_copy
ls -alhS
```

> ⚠ There is no trash can in the linux shell
>
> `rm` deletes permanently. There is no undo.

Wildcards. Often you want a command to act on many files at once: every `.csv` file, or every file whose name starts with `file_`. Instead of typing each name, you type a **pattern**, a short description of the names you mean, and the shell works out which files match. Pattern languages turn up everywhere in data work. The shell's version, called **wildcards** or **globs**, has just three building blocks: `*`, `?` and `[...]`. Later in this lecture, `grep` will search *inside* files using a richer pattern language called **regular expressions**. The two look alike and are easy to confuse: in a wildcard `*` means "any characters", while in a regular expression it means "repeat the previous thing any number of times". For now, wildcards:

```
touch tmp.csv file_1.txt file_2.txt file_10.tex   # touch takes several names
ls *.csv            # * matches any run of characters, including none
ls file_?.txt       # ? matches exactly one character: file_1.txt, not file_10
ls file_[12].txt    # [...] matches one character from the set
ls file_[0-9].*     # ranges work too plus other wildcards (the suffix can be anything)
ls file_[!1]*       # [!...] matches one character NOT in the set
ls *_1*             # wildcards can go anywhere in the name
ls file_{1,10}.*    # {a,b} is not a wildcard: the shell writes out both names
echo file_{1,10}.*  # see what a pattern expands to before using it with rm
```

The shell expands a pattern into the matching names *before* the command runs, so `ls *.csv` is really `ls covid.csv tmp.csv`, and `ls *` alone lists the contents of every subdirectory too. Patterns never match names that start with a dot (hidden files) unless you write the dot yourself.

Finding files: `find` walks a directory tree and prints every file that passes the tests you give it. The first argument is where to start; everything after it is a test.

```
find . -name "*.csv"         # find CSV files in the working directory and its subdirectories
find ~ -name "*.csv"         # all csv files anywhere under my home directory and its subdire
                             # (your list will differ; I have a lot of files in my WSL as I us
find . -type f               # only files...
find . -type d               # ...or only directories
find ~ -maxdepth 1 -type f   # don't descend into subdirectories
find . -size +10M            # files bigger than 10 MB
find . -size -1k             # files smaller than 1 KB (the ones we touched)
find . -empty                # empty files and directories
find . -mmin -30             # modified in the last 30 minutes
find . -mtime +7             # modified more than 7 days ago
find . -newer covid.csv      # modified after covid.csv was
```

Several tests in a row must all hold. Use `-o` for "or" and `-not` to negate. `find` is like its own mini programming language.

```
find . -type f -name "*.csv" -size +1M    # all three must hold
find . -name "*.csv" -o -name "*.tex"     # either suffix
find . -type f -not -name "*.csv"         # everything except csv files
```

Once `find` has the files, it can do something with them. `-exec` runs a command on each result, with `{}` standing for the file name:

```
find . -name "*.txt" -delete              # delete every match (no confirmation given)
ls -alhS
rm -i *.tex                               # ask before each deletion
```

Searching from `/` reaches directories you are not allowed to read, which produces error messages.

```
find / -name "*.csv*"
```

Every program has two separate output channels:

- stdout (channel 1) carries normal results, such as the file names find prints.
- stderr (channel 2) carries error messages, such as "find: '…': Permission denied."

Both go to your screen by default, so they look the same. They are separate streams, though, and the program chooses which one to write each line to. find writes matches to stdout and complaints to stderr. So we want to filter out the errors

```
find / -name "*.csv*" 2>/dev/null   # no more clutter from looking at useless directories
```

In 2>/dev/null, the 2> means "redirect channel 2 somewhere else", and /dev/null is a special file that throws away anything written to it. Everything on channel 2 is discarded, and channel 1 is untouched.

Redirects. A **redirect** sends a command's input or output somewhere other than the keyboard and screen, usually a file. The symbols are:

- `>` (send stdout to a file),
- `>>` (add stdout to the end of a file),
- `2>` (send the stderr to a file) and
- `<` (receive input from a file).

Demos:

```
echo "hello"                  # prints hello on the screen
echo "hello" > notes.txt      # redirect into a file so nothing printed to terminal because it
ls -alh
cat notes.txt                 # cat is "concatenate" which is used mostly to not concatenate h
echo "again" >> notes.txt     # >> adds the object of redirection to a new line at the end (i
wc -l notes.txt
cat notes.txt
echo "oops" > notes.txt       # > replaces the contents: no warning!
cat notes.txt
```

Redirects work with any command, and a redirected file is created if it doesn't exist:

```
ls > listing.txt              # save a directory listing
cat listing.txt
date > today.txt              # save the current date and time
cat today.txt
cat notes.txt notes.txt > twice.txt   # join files into a new file (the original intent of th
cat twice.txt
ls /nonexistent
ls /nonexistent > err.txt   #can we redirect this output?
cat err.txt                 #no, because redirect by default is stdout, not stderr, it's shor
ls /nonexistent 2> err.txt   # save the error message instead of showing it
cat err.txt
ls . /nonexistent
ls . /nonexistent > out.txt
cat out.txt
ls . /nonexistent 1> out.txt
cat out.txt
ls . /nonexistent 1> out.txt 2> err.txt      # results and errors in two different files
cat out.txt
cat err.txt
ls . /nonexistent > all.txt 2>&1      # results and errors in one file the &1 says direct to
cat all.txt
```

More about `cat`... Without file argument(s) it takes input from the keyboard:

```
cat                           # start typing... until CTRL+D
cat > keyboard_input.txt      # take the keyboard input as stdin and redirect it as stdout
ls -alh
cat keyboard_input.txt
```

First we will introduce multiline strings. They begin with an arbitrary string which denote the end of input. This is usually 'EOF' (for "end of file"). Thus EOF terminates the input. The "«" is a redirection in the opposite direction from before. It tells cat to take string input and not from the keyboard.

```
cat << 'EOF'
line 1
line 2
line 3
EOF
```

Now the following would make sense, but it ruins the "EOF" delimiter

```
cat << 'EOF' ####don't run this
line 1
line 2
line 3
EOF > stream_input.txt
```

So we can direct the output of cat as the first argument

```
cat > stream_input.txt << 'EOF'
line 1
line 2
line 3
EOF
ls -alh
cat stream_input.txt
```

A first shell script. A shell script is a text file of commands that the shell runs one after another, so a sequence you would otherwise retype can be saved and rerun. Here is the smallest useful one: a file `my_script.sh` with a single line, `touch file1.txt`, which creates an empty file.

```
echo "touch file1.txt" > my_script.sh   # sh is the standard extension for "shell script"
                                        # although it is not necessary to name it so, it's a
cat my_script.sh                        # touch file1.txt
```

Now let's run this script by giving it to "bash" (Bourne Again SHell) which just runs each line:

```
bash my_script.sh
ls -alhS
```

Let's learn some of basic bash syntax: variables (name="Ada"), if/for/while, &&, $(…), functions and comments (#). You can also use these at the linux prompt. We will upgrade `my_script.sh` into a short tour of the main bash features.

```
cat > my_script.sh << 'EOF'
#!/usr/bin/env bash
# this is a comment
# but that first line is not a comment. It is called the "shebang" (scroll below)
name="HUJI"                             # a variable: no spaces around =
echo "Hello, $name"                     # $name is replaced inside " "
echo 'Hello, $name'                     # but not inside ' '
echo "Today is $(date +%F)"             # $(...) pastes in a command's output "date +%F" prints
echo "This script is called $0"         # $0 is the script's own name, it's preset special vari
echo "I got $# arguments; the first is $1"   # $# counts them, $1 is first, another preset sp
if true; then                           # the simplest if: true always succeeds
  echo "trivial if statement"
fi                                      # close out the if-else with "fi"
if [ -f file1.txt ]; then               # test: does the file exist? (see [...] explanation bel
  echo "file1.txt exists"
else
  echo "file1.txt is missing"
fi
for i in 1 2 3; do                      # repeat for each item in a list
  echo "round $i"
done
greet() {                               # declare a function: a named block of commands, argume
  echo "Hi, $1!"                        # if there's a first argument
}
greet
greet Bob
greet Bob Alex
EOF
```

That creates the multiline file from the prompt. Now we run it

```
bash my_script.sh                       # run it
bash my_script.sh 52002 52019           # run it with two arguments: $1 $2 ...
```

What is `[ -f file1.txt ]`? It is not the special `if` condition syntax. It is the same as the `test` command in linux. This command returns to an "exit status" like a program, not to the screen (more on this later). It is an ordinary command, made of four separate words:

Piece	What it is
`[`	The name of a command (the same one as `test`). It is a real program, and also built into bash.
`-f`	An option meaning "is it a regular file that exists?"
`file1.txt`	The argument: the file to check.
`]`	The last argument. It does nothing except close the pair of brackets.

Because `[` is a command, every piece needs spaces around it. The command prints nothing; it only sets its exit status (0 if the test is true, 1 if not), and `if` simply runs whatever command follows it and checks that status. This is also why `if true` works: `true` is also a command that always exits 0.

```
[ -f file1.txt ]              # nothing is printed! But that doesn't mean it didn't work
echo $?                       # $? stores the exit status (in our case, 0 if the file exists
[ -f file1.txt ]; echo $?     # two linux commands on one line, separated by a semicolon
[ ! -f file1.txt ]; echo $?   # negation of [ -f file1.txt ]
test -f file1.txt; echo $?    # identical: test is [ without the ]
[-f file1.txt]                # error: command not found (no space after [)
[ -f file1.txt]               # error: missing ']' (no space before ])
[ -d results ]; echo $?       # is it a directory?
[ -s covid.csv ]; echo $?     # exists and is not empty?
[ "$a" = "$b" ]; echo $?      # are the strings equal? (spaces around = required). Fails beca
[ $n -gt 5 ]; echo $?         # is n greater than 5? Fails because it doesn't have context fo
```

The shebang. The first line, `#!/usr/bin/env bash`, is called the **shebang** (from "sharp" `#` and "bang" `!`). It tells the system which program should run the file when you execute it directly, as we will see below. For a Python script it would be `#!/usr/bin/env python3`, and for an R script `#!/usr/bin/env Rscript`. A few details:

- It must be the very first line of the file: no blank line or space before it.
- `env bash` means "find `bash` wherever it is installed on this computer". Executable programs need absolute paths. That is more portable than writing a fixed path like `#!/bin/bash`, because the location differs between systems (for example on a Mac with a newer bash from Homebrew).
- When you run `bash my_script.sh`, you already chose the program yourself, so bash treats the line as an ordinary comment, because it starts with `#`.

Now run the script as an "executable program" instead of handing it to bash. This has a few advantages * you don't need to know what program executes the code inside (the shebang handles it) * you can set this program in PATH and access it globally * other programs can call this program easily

Let's try it:

```
./my_script.sh                          # error: Permission denied (see below)
```

The last line fails because the file is not marked as something the computer may run. That is what permissions control, and we fix it next. (We write a more useful script later in this lecture.)

Permissions. In the output of `ls -l`, the first column has a string which is always 10 characters: one for the type of item, then three groups of three. Then there are two strings which tell you who is the owner and which group has rights

```
ls -alh
```

What do these characters mean? The following is the default permissions:

```
- rw- r-- r--
| |   |   |
| |   |   +-- 8-10: permissions for everyone else ("other")
| |   +------ 5-7:  permissions for the group (we won't go into this)
| +---------- 2-4:  permissions for the owner (the user who owns the file)
+------------ 1:    the type of item (`-` for ordinary file, `d` for directory, `l` for link
```

Am I "kapelner"?

```
whoami
```

When using the prompt you are a user and that user has certain permissions for each file and directory. For "real" servers, there are tons of users on the same system, so security becomes more important.

The three letters of each group always come in the order `r`, `w`, `x` and `-` means not granted.

	on a file	on a directory
`r`	read the contents (`cat`, `head`)	list the names inside (`ls`)
`w`	change the contents	create, delete and rename files inside
`x`	run it as a program	enter it (`cd`) and reach the files inside

on a file	on a directory

Some strings you will meet:

String	What it means
`-rw-r--r--`	an ordinary data file: the owner can read and write, everyone else can only read
`-rwxr-xr-x`	a ordinary program file: everyone can run it, only the owner can change it
`drwxr-xr-x`	an ordinary directory: everyone can enter and list its files, only the owner can change it
`-rw-------`	a private file: only the owner can touch it (for passwords and keys)

The file `my_script.sh` we just created is `-rw-r--r--`. Nobody has an `x`, which is exactly why `./my_script.sh` said `Permission denied`. We need to "change its mode" via `chmod`

```
ls -lha
chmod +x my_script.sh   # make it executable for everyone:
ls -lha
./my_script.sh          # now it executes
```

If this is my private script, then maybe I want to make sure nobody else has access to it:

```
chmod 600 my_script.sh  # the "600" controls the locations of the permissions - read about it
                        # I've memorized a few of these numeric codes
ls -lha                 #now only i can read and write
chmod u+x my_script.sh  #and only i can execute
ls -lha
./my_script.sh          # now executes
```

If this can be used and edited by everyone (not usually recommended), then

```
chmod 777 my_script.sh
ls -lha
```

Let's talk about the root user (or "super" user, the administrator of the machine). You will definitely need to know about this. For example:

```
ls -l /etc/security/limits.conf
```

This file controls resource limits and may need to be changed for certain big data operations. It makes sense only the server admin should be able to change this. Look at the owner and the permissions. What does this grant me?

Let's try adding my setting to it

```
echo "my special new line" >> /etc/security/limits.conf
```

Fail. I don't have rights. Can I give myself access?

```
chmod a+w /etc/security/limits.conf
```

Fail. I'm not the owner. We will return to this later.

Editing in the terminal

On a remote server there is no graphical editor, so you need to be able to edit a file in the terminal. The two common terminal editors are `vi` and `nano`. We will only use `vi`; you can learn `nano` on your own.

> **Which is used more, vi or nano?**
>
> Vi, by about two to one. In the 2025 Stack Overflow Developer Survey, 24.3% of developers said they use Vim and 12.2% use Nano. Another 14% use Neovim, a modern rewrite of Vim, so the vi family together is ahead by more than three to one. Also, you can count on finding vi. It's installed on virtually every Unix system, including bare servers and containers where nothing else is installed.

However, it has a learning curve. It has **modes**, which is what confuses everyone at first.

```
vi covid.csv
```

The survival kit:

Keys	What it does
`:<cmd>` +Enter	Run a `vi` command
`:set nu` +Enter	Add line numbers

Keys	What it does
`i`	Enter insert mode: now you can type
Esc	Leave insert mode
`:767` +Enter	Go to line number 767
`gg`	Go to the top of the file (in insert mode)
`G`	Go to the bottom of the file (in insert mode)
Ctrl+F	Go forward = down one page (in insert mode)
Ctrl+B	Go back = up one page (in insert mode)
`:w` +Enter	Save
`:q` +Enter	Quit
`:wq` +Enter	Save and quit
`:q!` +Enter	Quit **without** saving
`/word` +Enter	Search for "word" (`n` for next match)
`d3`	Delete 3 lines (`dd` a/lone deletes one)
`u`	Undo

If you're ever stuck in `vi`, press Esc a few times, then type `:q!` and Enter.

> Practice later
>
> `vimtutor` is a 30-minute interactive vi tutorial. On Ubuntu, install it with `sudo apt install vim`.

Back to permissions. What if I need to edit a file that only the administrator can edit?

```
vi /etc/security/limits.conf
```

You need to use "sudo" (i.e. super user do)

```
sudo vi /etc/security/limits.conf
```

Now of course, you need to actually *be* a super user to do this. That is, another super user gave it to you, or you set up the machine yourself. If its your MAC or you set up WSL, you are super user. Later in the semester, when we login to the university's machines, maybe not, unless they're giving you a carved out virtual environment. We will see later as I don't know how they set up their machines.

You've seen `sudo` during your setup. You used it to update and install programs e.g. this will go to ubuntu's servers (and whatever other servers that catalog programs) and see if you have any updates available for what you have installed. The next line will do upgrades on anything that needs updating.

```
sudo apt update
sudo apt upgrade #if necessary
```

Looking at data without opening it

A file can be far larger than what you want to (or can) load into memory, but you can still look at it.

We will now return to our downloaded file. The columns are `date`, `county`, `state`, `fips` (a numeric code for the county), `cases` and `deaths`. Source: The New York Times, based on reports from state and local health agencies, https://github.com/nytimes/covid-19-data.

```
cat covid.csv          # print the whole file (bad idea for big files!)
less covid.csv         # scroll through a file (q quits, / searches)
head covid.csv         # first 10 lines
head -n 3 covid.csv    # first 3 lines
tail covid.csv         # last 10 lines
tail -n 3 covid.csv    # last 3 lines
```

Pipes (similar to redirection)

A redirect connects a command usually to a **file**. A **pipe** connects a command to **another command**. The symbol is |, called the pipe or the vertical bar. It sends the stdout of the command on its left into the stdin of the command on its right:

```
command1 | command2        # command1's output becomes command2's input
```

Compare the two ways of counting the first three lines of a file. With a redirect, you need to

```
head -n 3 covid.csv > first3.txt   # with a redirect: save to a file...
wc -l < first3.txt                 # ...then feed the file to the next command
rm first3.txt                      # clean up after yourself!
```

Much easier with a pipe command:

```
head -n 3 covid.csv | wc -l        # with a pipe: both steps at once, no file
#^^^^^^^^^^^^^^^^^^   ^^^^^
#     producer        consumer
```

A few more small ones:

```
ls | wc -l                    # how many items are in this folder?
ls -l | head -n 3             # only the first 3 lines of a long listing
echo "hello world" | wc -w    # 2: the text from echo became wc's input
```

You can chain as many commands as you like. Each one does one small job:

```
tail covid.csv | sort | uniq
```

Pipes process data as a **stream**: each command reads a line, handles it, and passes it on. None of these commands loads the whole file into memory, which is why they work on files much bigger than your RAM. This is the idea behind the one-pass algorithms (which we may see later). For example:

```
cat covid.csv | head
```

`cat` does not load the file. It reads a block (around 128 KB), sends it to the pipe, and repeats. Its memory use is tiny, however large the file is. Above when we ran `cat covid.csv`, it started printing immediately. If it had to load the whole file first, nothing would appear until the load finished.

A pipe has a small buffer, and that is what keeps it streaming. On Linux the pipe holds about 64 KB. When it is full, the producer (the command on the left) is paused until the consumer (the command on the right) catches up. This is called backpressure. So even a 100 GB file can usually pass through `cat | ... | ...` using a few megabytes in total, with all the commands running at the same time.

Once `head` quits, cat gets a "broken pipe" signal and stops, so it never bothers to read from the rest of the file.

Here's a nice example of pipe usage. To read a CSV comfortably, line up the columns with the `column` command:

```
head covid.csv | column -s, -t    # -s, : fields are separated by commas, -t : table format
```

Searching with grep

`grep` (global regular expression print) prints the lines of its input argument that match a pattern. We will demo this using our dataset as the input argument and use "| head" to limit the results shown to screen or "| wc -l" to count.

```
grep "Alaska" covid.csv | head -n 4     # lines containing Alaska
grep -i "alaska" covid.csv | head -n 4  # -i: ignore upper/lower case
grep -v "Alaska" covid.csv | head -n 4  # -v: lines that do NOT match
grep -c "Alaska" covid.csv              # -c: count matching lines
grep -vc "Alaska" covid.csv             # options combine: count the others
grep -n "Alaska" covid.csv | head -n 4  # -n: show line numbers
grep -m 2 "Alaska" covid.csv            # -m 2: stop after 2 matches (the color highlighting
grep -o "2020-03-[0-9][0-9]" covid.csv | head -n 2   # -o: print out only the match componen
grep -c "Ada" covid.csv                 # compare: also counts Adams, Adair
grep -m 1 -C 2 "Anchorage" covid.csv    # -C 2: 2 lines of context
```

You can search for several things at once, and in many files:

```
grep -e "Alaska" -e "Hawaii" covid.csv | wc -l    # -e: either pattern
grep -f pats.txt covid.csv | wc -l      # -f: patterns listed in a file
grep -F "." covid.csv | wc -l           # -F: literal text, not a pattern
grep -l "Alaska" sample.csv covid.csv   # -l: only NAMES of files that match
grep -c "Alaska" sample.csv covid.csv   # a count for each file
grep -r "Cook" . | head -n 3            # -r: search every file under here
grep -rl "Cook" .                       # ...and just list which files
grep -r --include="*.csv" -l "Cook" .   # ...but only in csv files
```

`grep` also reports whether it found anything, through its **exit status**, which every command has: 0 means success, anything else means failure. The special variable `$?` holds the status of the last command:

```
grep "Alaska" covid.csv > /dev/null; echo $?    # 0: found
grep "Atlantis" covid.csv; echo $?              # 1: not found
grep -q "Alaska" covid.csv; echo $?             # -q: quiet, only the status
```

The exit status is what scripts use to make decisions.

`grep` after another command. So far `grep` read a file. It can also read the output of any other command through a pipe, which makes it a filter: the first command produces a lot of output, and `grep` keeps only the lines you care about. When no file name is given, `grep` reads its input from the pipe.

```
ls | grep csv                       # only the names containing csv
ls -l | grep "^d"                   # only directories: their line starts with d
ls -l | grep -v "^d"                # -v: everything except directories
find . -name "*.csv" | grep covid   # csv files whose path also contains covid
```

```
find . -type f | grep -v "\.git/"   # all files, but skip anything inside .git
df -h | grep -v tmpfs               # disk usage without the tmpfs lines
env | grep "^HOME"                  # one environment variable
history | grep chmod                # find that command you ran last week
```

Notice that the first command does the broad work and `grep` narrows it down. You can keep going, e.g. `find . -type f | grep csv | grep -v old | wc -l` counts the csv files that are not "old".

Regular expressions (for you at home)

A **regular expression** (regex) is a pattern that describes a set of strings. We have been using the simplest regex, a plain word that matches itself. The real power comes from special characters. A few meet you almost everywhere (R, Python, SQL, text editors), so learning them pays off all semester.

Pattern	Matches	Example
`.`	any one character	`Co.k` matches Cook
`*`	the previous thing, zero or more times	`Co*k` matches Ck, Cok, Coook
`+`	the previous thing, one or more times	`Co+k` matches Cok but not Ck
`?`	the previous thing, zero or one times	`Lou?isiana` matches Loisiana and Louisiana
`^`	the start of the line	`^2020-03`
`$`	the end of the line	`,0$`
`[abc]`	one character from the set	`[BC]ook`
`[a-z]`, `[0-9]`	one character from a range	`[0-9][0-9]`
`[^,]`	one character NOT in the set	`[^,]*` is "a field"
`{n}`, `{n,m}`	repeat exactly n, or n to m, times	`[0-9]{5}`
a|b	either a or b	Texas|Ohio
`( )`	group things together	(Texas|Ohio),
`\.`	a literal special character	`\.` is a real dot

Use `grep -E` ("extended") whenever your pattern has `+ ? | ( ) { }`, otherwise `grep` wants a backslash before each of them. Also put patterns in **single quotes**, so the shell doesn't try to interpret characters like `*`, `$` or `|` before `grep` sees them.

Anchors first, `^` and `$`:

```
grep '^2020-03-15' covid.csv | head -n 2       # lines that START with a date
grep -c ',0$' covid.csv                        # lines that END with ,0
grep -c '^2020-0[1-3]' covid.csv               # January to March: a set
grep -Ec '^2020-(03|04)-' covid.csv            # March or April: alternatives
grep -E '^2020-03-1[0-5],' covid.csv | wc -l   # March 10 to 15
grep -x 'date,county,state,fips,cases,deaths' covid.csv   # -x: whole line
```

Repetition and wildcards inside a pattern:

```
grep -c 'New .*City' covid.csv          # .* is "anything": New ... City
grep -E 'o+k' covid.csv | head -n 1     # one or more o, then k: Cook
grep -E 'Lou?isiana' covid.csv | head -n 1    # the u is optional
grep -E 'a{2}' covid.csv | head -n 1    # a double "aa"
grep -E ',[0-9]{6,},[0-9]+$' covid.csv | head -n 2    # 100,000 cases or more
```

Precision: the `[^,]*` idiom means "any run of characters that isn't a comma", i.e. one CSV field. It lets you say which column you mean:

```
# columns: date , county , state , ...  so the state is after two commas
grep -Ec '^[^,]*,[^,]*,Texas,' covid.csv    # only rows whose STATE is Texas
grep ',Texas,' covid.csv | grep -v '^[^,]*,[^,]*,Texas,' \
  | cut -d, -f2,3 | sort -u
```

The last line shows the 495 impostors. Missing values are easy to find as well. The `fips` column is empty for New York City (the data combines its five boroughs) and for the rows marked "Unknown", so those rows contain two commas in a row:

```
grep -c ',,' covid.csv                        # rows with an empty field
grep ',,' covid.csv | head -n 2               # which ones?
grep -E '^[0-9]{4}-[0-9]{2}-[0-9]{2},' covid.csv | wc -l    # valid dates
grep -Ev '^[0-9]{4}-[0-9]{2}-[0-9]{2},' covid.csv           # the odd ones out
grep -E '^2020-03-15,(New York City|Baldwin),' covid.csv    # a group
grep -E 'Cook,(Illinois|Georgia)' covid.csv | cut -d, -f3 | sort | uniq -c
```

The last two lines of that block show why `grep` is a data-cleaning tool: the fourth finds rows that do *not* look like data (here, the header), which is how you spot problems in a file too big to read. The same idea works with files that have comment lines:

```
printf '# settings\nname=x\n\n# more\nn=3\n' > cfg.txt
grep -v '^#' cfg.txt                  # drop the comment lines
grep -c '^$' cfg.txt                  # ^$ is an empty line: count them
grep -v -e '^#' -e '^$' cfg.txt       # drop comments AND blank lines
```

> **Wildcards vs. regular expressions**
>
> Both are patterns, and the characters look alike, but they are different languages. In the shell, `*.csv` means "any name ending in .csv". The same thing as a regex is `.*\.csv$`: `.` is any character, `*` repeats it, `\.` is a real dot, and `$` is the end. In particular, the wildcard `*` stands alone, while the regex `*` repeats the thing before it. The shell uses wildcards for file *names*, and `grep` uses regular expressions for text *inside* files.

`tail -f` keeps showing new lines as they are added to a file, which is how you watch a log of a long-running job. To try it, open a second terminal and run the first line, which adds the date to `logfile.txt` every second. Then, in the first terminal, run the second line. Press Ctrl+C in both when done.

```
cd ~/workspace/HUJI_52002_Fall_2026/scratch/lec01    # terminal 2
while true; do date >> logfile.txt; sleep 1; done    # terminal 2
tail -f logfile.txt                                  # terminal 1
```

What if we forget to close terminal 2?

Finding and stopping a process

Every running program is a **process** with an ID number (its **PID**). Let's open a third terminal. The command `ps` lists processes:

```
ps                    # your processes in this terminal (not super interesting)
                      # note "bash" is running - it's the program actually receiving your pr
ps -u $USER | head    # just your processes, from every terminal
ps aux | head         # every process on the machine
```

In `ps aux`, `a` means processes of all users, `u` means a user-oriented format (with the owner, PID, `%CPU`, `%MEM` and command) and `x` includes programs that have no terminal. The list is long, so we use `head`. To find one process by name, filter the list with `grep`:

```
ps aux | grep sleep   # find a process by name (grep filters the list)
pgrep -a sleep        # the same, simpler: PID and full command of matches
```

The first command prints something like this (your numbers will differ):

```
kapelner   4821  0.0  0.0  17664  8064 pts/0  S+  18:02  0:00 sleep 1
kapelner   4830  0.0  0.0   6716  2560 pts/2  S+  18:02  0:00 grep sleep
```

The second line is the `grep` command itself, because its own command line contains the word "sleep". `pgrep` leaves itself out, which is one reason to prefer it.

Why did we search for `sleep` and not for `while`? There is no program called `while`. The loop is part of the bash language, so it is run by the shell itself, the `bash` in terminal 2. What you can see is the loop's children: every second it starts a new `sleep 1`, which exits after one second, so its PID is different each time you look. What stays the same is the **parent**. Ask `ps` for the parent PID (PPID) of each process, and then look up the parent:

```
ps -axo pid,ppid,command | grep "[s]leep"   # PID, parent PID, command
#  4821  4702 sleep 1                      # the loop's shell is process 4702
ps -p 4702                                 # who is process 4702?
#    PID TTY          TIME CMD
#   4702 pts/0    00:00:00 bash
```

The `[s]leep` is a pattern that matches the word "sleep" but not its own text, so `grep` doesn't list itself. The `-axo` form of `ps` works on both Linux and a Mac.

To watch processes live instead, use `htop`, and press `q` to quit:

```
htop                  # live view of every process, CPU and memory
top                   # the older version, available everywhere
```

On Ubuntu, install `htop` with `sudo apt install htop`. On a Mac, `top` is already there, `brew install htop` gets `htop`, and the Activity Monitor app shows the same information in a window. In `htop`, F4 filters the list (type `sleep`), F5 shows the processes as a tree (the loop's `bash` with its `sleep` below it), F9 sends a signal to the selected process, and `q` quits.

Stopping a process. The command `kill` takes a PID. Despite its name, it sends the process a **signal**, and the default signal is a polite request to stop, which lets the program clean up (close files, remove temporary data):

```
kill 4702             # ask process 4702 to stop (signal SIGTERM)
kill -9 4702          # force it to stop at once (SIGKILL): last resort
pkill sleep           # stop every process named sleep
pkill -f long_job.py  # -f: match the whole command line, not just the name
```

`kill -9` cannot be refused, but the program gets no chance to clean up and may leave half-written files behind, so try plain `kill` first. You can only stop your own processes. Someone else's, or root's, needs `sudo`.

Our loop teaches two lessons about this:

- **Stop the parent, not the child.** `pkill sleep` stops the current `sleep 1`, but the loop immediately starts a new one. The loop only ends when its shell stops.
- **Interactive shells ignore the polite request.** `kill 4702` on the shell in terminal 2 appears to do nothing, because an interactive shell ignores SIGTERM. `kill -9 4702` works, but it ends the shell itself, and terminal 2 usually closes. The normal way to stop a loop is **Ctrl+C in its own terminal**, which sends the process a different signal, SIGINT (number 2).

> Try it
>
> In terminal 2 run `sleep 300`, which blocks the prompt. In terminal 3 run `pgrep -a sleep` to see its PID, then `kill` that PID. Terminal 2 prints `Terminated` and gives the prompt back. Then run it again and stop it with `kill -9`, and compare.

Columns, sorting and counting

`cut` picks columns, `sort` orders lines, and `uniq` collapses repeated lines. They are the backbone of command-line data analysis:

```
cut -d, -f1,3 covid.csv | head -n 2     # columns 1 and 3
cut -d, -f2-4 covid.csv | head -n 2     # columns 2 through 4
cut -d, -f4- covid.csv | head -n 2      # column 4 to the end
cut -c1-7 covid.csv | head -n 2         # characters 1-7: not columns at all
cut -c1-7 body.csv | sort | uniq -c     # rows per month (2020-01 etc.)
```

> ⚠ `cut` doesn't understand quoted fields
>
> `cut -d, -f2` splits at *every* comma, even one inside quotes. For a file with a line `"Smith, John",Boston`, column 2 comes out as `John"` instead of `Boston`. `covid.csv` has no quoted fields, so we are safe here, but real CSV files often do. Check with `grep -c '"'` first, and use a tool that understands CSV (R, Python, DuckDB) when it matters:
>
> ```
> printf 'name,city\n"Smith, John",Boston\n' > quoted.csv
> cut -d, -f2 quoted.csv
> ```

`sort` has many options:

```
printf '10\n9\n100\n' | sort          # 10, 100, 9: text order, not numeric!
printf '10\n9\n100\n' | sort -n       # 9, 10, 100: -n sorts as numbers
printf '10\n9\n100\n' | sort -nr      # 100, 10, 9: -r reverses
printf 'b\nA\na\nB\n' | sort          # a A b B: lower and upper case mix
```

```
printf 'b\nA\na\nB\n' | sort -f       # -f: ignore case when comparing
printf '1K\n2G\n3M\n' | sort -h       # 1K, 3M, 2G: -h understands K, M, G
sort -u body.csv | wc -l              # -u: sort, dropping duplicate lines
sort -R body.csv | head -n 3          # -R: shuffle the lines (random rows)
printf 'x\ny\nz\n' | sort -c; echo $? # -c: just check if sorted; 0 = yes
```

To sort a CSV by a column, tell `sort` the separator with `-t` and the column with `-k`. Write the column twice, `-k5,5`, so it sorts on that column alone and not everything after it:

```
sort -t, -k5,5 -n -r body.csv | head -n 3   # largest case counts, any county
sort -t, -k3,3 -k5,5nr body.csv | head -n 3 # state, then cases (high to low)
sort -t, -k4,4 -u body.csv | wc -l          # one row per distinct fips code
```

`uniq` removes repeated lines, but only repeated *neighbors*, which is why it almost always follows a `sort`:

```
printf 'a\nb\na\na\nc\n' | uniq               # a b a c: neighbors only
printf 'a\nb\na\na\nc\n' | sort | uniq        # a b c
printf 'a\nb\na\na\nc\n' | sort | uniq -c     # with counts: 3 a, 1 b, 1 c
printf 'a\nb\na\na\nc\n' | sort | uniq -d     # -d: only lines that repeat
printf 'a\nb\na\na\nc\n' | sort | uniq -u     # -u: lines that appear once
printf 'a\nA\n' | sort | uniq -ic             # -i: ignore case
```

Put them together and you can answer real questions about a file with 885,000 rows in a few seconds:

```
cut -d, -f3 body.csv | sort | uniq -c | sort -rn | head -n 5   # top 5 states
cut -d, -f3 body.csv | sort -u | wc -l           # how many distinct "states"?
cut -d, -f2,3 body.csv | sort -u | wc -l         # distinct county-state pairs
cut -d, -f1 body.csv | sort -u | head -n 1       # the first date in the file
grep -E ',(Texas|Ohio),' body.csv | cut -d, -f1,2,5 | head -n 2
```

The pattern `sort | uniq -c | sort -rn | head` is the single most useful idiom here: *count how often each value occurs, and show the most common first.* Notice that `cut -d, -f3 | sort -u | wc -l` says 55, not 50: the file includes DC, Puerto Rico, Guam and other territories.

Try it

1. Which five days have the most rows? (`cut`, `sort`, `uniq -c`, `sort -rn`, `head`)
2. Which county has the largest number of deaths, ever? (Hint: `sort` by column 6.)
3. How many distinct county names are there? Why is it different from the number of distinct (county, state) pairs?

Shell scripts

A **script** is a text file of commands that the shell runs one after another. Anything you type at the prompt, you can save in a script and rerun. Before we write one, we need a few pieces of the shell's own language.

Variables and quoting

```
name="Ada"                    # make a variable: NO spaces around the =
echo $name                    # use it with a $
echo "Hello, $name"           # inside double quotes, $name is replaced
echo 'Hello, $name'           # inside single quotes, nothing is replaced
echo "Today is $(date +%F)" # $(...) runs a command and pastes in its output
echo "files here: $(ls | wc -l)"
echo "$(wc -l < covid.csv) rows"
```

Quotes also protect spaces. A name with a space is two words to the shell unless you quote it:

```
touch "my file.txt"           # one file with a space in its name
ls my file.txt                # error: looks for files "my" and "file.txt"
ls "my file.txt"              # works
ls my\ file.txt               # works: a backslash escapes the space
```

The habit to build: **put double quotes around every variable** (`"$name"`, `"$file"`), because a filename with a space will otherwise break your script.

Chaining commands and exit status

Recall that every command ends with an exit status: 0 for success, anything else for failure. The shell can chain commands based on it:

```
echo a; echo b                    # ; runs both, whatever happens
mkdir results && cd results       # && : second only if the first worked
cd /nonexistent && echo "arrived" # the echo never runs
false || echo "first one failed"  # || : second only if first failed
grep -q "Alaska" covid.csv && echo "found it"
grep -q "Atlantis" covid.csv || echo "not there"
true; echo $?                     # 0
false; echo $?                    # 1
```

Your first script

Let's write a script that counts the rows for one state. The next command creates the file `count_state.sh` with the text between the two `EOF` lines (this is a "here document", a way to type a file into the terminal; you could equally write the file with `vi`):

```
cat > count_state.sh << 'EOF'
#!/usr/bin/env bash
# Usage: ./count_state.sh FILE STATE
if [ $# -lt 2 ]; then
  echo "usage: $0 FILE STATE" >&2
  exit 1
fi
file=$1
state=$2
n=$(grep -Ec "^[^,]*,[^,]*,$state," "$file")
echo "$state has $n rows in $file"
EOF
```

What is in it:

- `#!/usr/bin/env bash` on the first line (the "shebang") says which program should run the file.
- Lines starting with `#` are comments.
- `$1`, `$2`, … are the **arguments** you type after the script's name. `$#` is how many there are, `$0` is the script's name, and `"$@"` is all of them.
- `if [ ... ]; then ... fi` runs commands only when a test is true. Here: "if there are fewer than 2 arguments". The spaces inside `[ ]` are required.

- `>&2` sends the message to stderr, and `exit 1` stops with a failure status.
- Everything else is commands you already know.

There are two ways to run it:

```
bash count_state.sh covid.csv Texas     # run it through bash explicitly
./count_state.sh covid.csv Texas        # error: Permission denied
ls -l count_state.sh                    # no x: not executable
chmod +x count_state.sh                 # add execute permission (x)
ls -l count_state.sh                    # -rwxr-xr-x: now it has x
./count_state.sh covid.csv Texas        # works
count_state.sh covid.csv Texas          # error: command not found
./count_state.sh covid.csv              # too few arguments: prints the usage
echo $?                                 # 1: the script's exit status
./count_state.sh covid.csv Utah 2>/dev/null   # hide any error messages
```

The `./` means "the file in this directory". Without it, the shell looks for the name only in the directories of your `PATH` (which we met earlier), and the current directory isn't among them. See the last section below for how to put your scripts on the `PATH`.

Loops and tests

A **for loop** repeats commands for every item in a list. This is how you run the same analysis on many states, files or months:

```
for i in 1 2 3; do echo "round $i"; done          # a fixed list
for i in $(seq 1 5); do echo "n=$i"; done         # seq makes a sequence
for i in {1..5}; do echo "n=$i"; done             # a shortcut for the same
for s in Texas Ohio Utah; do                      # a loop can span lines
  echo "$s: $(grep -Ec "^[^,]*,[^,]*,$s," covid.csv)"
done
for m in 01 02 03; do echo "$m $(grep -c "^2020-$m" covid.csv)"; done
for s in Texas Ohio Utah; do                      # a file for each state
  grep -E "^[^,]*,[^,]*,$s," covid.csv > "$s.csv"
done
for f in *.csv; do echo "$f has $(wc -l < "$f") lines"; done   # all csv files
head -n 2 covid.csv | while read line; do echo "got: $line"; done
```

The `if` test can check many things. The brackets `[ ... ]` are really a command called `test`, and the useful ones are:

Test	True when
`[ -f covid.csv ]`	the file exists
`[ -d results ]`	the directory exists
`[ -s covid.csv ]`	the file exists and is not empty
`[ "$a" = "$b" ]`	the two strings are equal
`[ $n -gt 5 ]`	the number is greater than 5 (also `-lt -ge -le -eq -ne`)
`[ ! -f covid.csv ]`	the test inside is NOT true

```
if [ -f covid.csv ]; then echo "exists"; else echo "missing"; fi
[ -s Utah.csv ] && echo "Utah.csv has data"
n=7; if [ $n -gt 5 ]; then echo "big"; fi
if grep -q "Utah" covid.csv; then echo "Utah is in the file"; fi
if [ ! -d results ]; then mkdir results; fi
```

The last line of the block above shows how `if` works with any command, not just `[ ]`: it asks "did this command succeed (exit status 0)?" That is also why `grep -q` exists.

> Try it
>
> Write a script `top_states.sh` that takes a file and a number N, and prints the N states with the most rows. Use `$1` and `$2`, and the pipeline from the sorting section (`head -n "$2"`). Then make it print a usage message and exit if the file does not exist.

Your own commands: `PATH`, `export` and `alias`

The `PATH` is a list of directories. When you type a command name, the shell looks for it in each of them. A good place for your own scripts is a `bin` folder in your home directory:

```
mkdir -p ~/bin                          # a folder for your own programs
cp count_state.sh ~/bin/                # put the script there
export PATH="$HOME/bin:$PATH"           # add ~/bin to the front of the PATH
which count_state.sh                    # now the shell finds it...
count_state.sh covid.csv Utah           # ...and it runs from anywhere
```

`export` makes a variable available to the programs you start, and the `PATH` change lasts only as long as this terminal window. To make it permanent, add the line to the file that the shell reads whenever it starts, `~/.bashrc` on Ubuntu (`~/.zshrc` on a Mac). An **alias** is a shortcut for a longer command, and it belongs in the same file:

```
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc    # for every new terminal
echo "alias ll='ls -lha'" >> ~/.bashrc              # a shortcut: ll
source ~/.bashrc                      # read the file now, instead of reopening
type ll                               # what kind of name is ll?
```

Processes

We already met `ps`, `htop` and `kill` for finding and stopping a process (in "Finding and stopping a process" above). Here we look at how big the machine is, and at running things in the background.

How big is the machine you are on? This matters for deciding how much data fits and how many things can run in parallel:

```
nproc                 # number of CPU cores (on a Mac: sysctl -n hw.ncpu)
free -h                # memory used and free (Linux only)
uptime                # how long it has been up, and the recent load average
```

Demo: watch a process eat CPU. Run this in one terminal, and `htop` in another:

```
yes > /dev/null &     # a useless program that uses 100% of one core
```

The `&` runs it in the **background**, so you get your prompt back. Now manage it:

```
jobs                  # list background jobs in this terminal
fg                    # bring it to the foreground (then Ctrl+C stops it)
kill %1               # or: stop background job number 1
pkill yes             # or: stop every process named yes (or kill its PID)
```

Forgot the `&`? You can still get your prompt back. Run something slow in the foreground, such as `sleep 300`, and then:

```
# press Ctrl+Z   # pauses the running command and gives you the prompt back
bg               # lets the paused command continue, in the background
jobs             # it is listed as Running
fg               # bring it back to the foreground; Ctrl+C then stops it
```

A job started with `&` belongs to your terminal window and stops when you close it. For something long, like a model fit on a remote server, use `nohup` ("no hang-up"), which keeps it alive after you log out. It uses the redirections from earlier to save the output:

```
nohup python3 long_job.py > out.log 2>&1 &     # survives logging out
tail -f out.log                               # check on it whenever you like
```

Demo: watch a process use memory. With `htop` open, and `free -h` run before, during and after, run:

```
python3 -c "x = 'a' * 10**9; input('Press Enter to quit')"
```

Watch the memory bar jump by about 1 GB (the `used` column of `free -h` does too). Press Enter and watch it drop. This is what "fits in RAM" means in practice.

Wrap-up

Cheat sheet

Command	What it does
`pwd`, `ls`, `cd`	Where am I, what's here, go somewhere
`mkdir`, `touch`	Make a directory, make an empty file
`cp`, `mv`, `rm`	Copy, move/rename, delete (permanently!)
`*`, `?`, `[...]`	Wildcards: match many file names at once
`find`	Search for files by name, size, age, type, and act on them
`chmod`, `sudo`	Change permissions, run as administrator
`vi`	Edit files in the terminal
`cat`, `less`, `column`	Print a file, scroll through a file, line up columns
`head`, `tail`, `wc`	First lines, last lines, count lines/words/bytes
`du`, `df`, `file`, `stat`	Space used, free space, what a file is
`time`	How long a command takes
`>`, `>>`, `2>`, `<`, `tee`	Write, append, send errors, read input, copy a stream
`|`	Pipe: stdout of one command into stdin of the next
`grep -i -v -c -n -w -o -E -r -l`	Find lines that match a pattern or regular expression
`cut`, `sort`, `uniq`	Pick columns, sort lines (`-n`, `-r`, `-k`), count duplicates
`$?`, `&&`, `||`	Exit status, "then", "otherwise"
`$var`, `$(cmd)`, `for`, `if`	Variables, command output, loops, tests
`PATH`, `export`, `alias`, `source`	Where programs are found, settings, shortcuts, reload
`ps`, `htop`, `top`, `free`, `nproc`	See running processes, memory and cores
`&`, `jobs`, `fg`, `bg`, `kill`, `nohup`	Background, list, foreground, stop, survive logout
`man`, `--help`	Get help

Regular expression building blocks

Pattern	Matches
`. * + ?`	any one character; repeat 0+, 1+, 0 or 1 times
`^ $`	start of line, end of line
`[abc] [0-9] [^,]`	one of a set, one of a range, one NOT in the set
`{3} {2,5}`	repeat exactly 3 times, 2 to 5 times
(a|b)	a group with alternatives
`\.`	a literal special character

Check yourself

These are the kinds of questions you should be able to answer on an exam, without a computer or AI.

1. What's the difference between an absolute and a relative path? Give an example of each.
2. You're in `~/bigdata/lec01`. What does `cd ../..` do?
3. What's the difference between `>` and `>>`?
4. You opened a file in `vi` and typed text you don't want to keep. How do you get out without saving?
5. Why is `head` instant on a 10 GB file while `cat` is not?
6. What does `cut -d, -f3 covid.csv | sort | uniq -c` produce?
7. Your laptop's fan is suddenly loud. How do you find out which process is responsible, and how do you stop it?
8. Why is `rm -r *` dangerous?
9. `ls /nonexistent > out.txt` still prints an error. Why? Write a command that puts the normal output in `out.txt` and the errors in `err.txt`.
10. What is the difference between `2>&1 > all.txt` and `> all.txt 2>&1`?
11. What does `grep -v '^#' file` print? What does `grep -c '^$' file` count?
12. Write a `grep` that matches lines of `covid.csv` whose *state* is Ohio (and not a county named Ohio). Why doesn't `grep ",Ohio,"` do it?
13. What does `printf '10\n9\n100\n' | sort` print? How do you get numeric order?
14. Write a pipeline that prints the ten most common county names in `covid.csv`.
15. Why is `uniq` almost always used after `sort`?
16. `cut -d, -f2` on the line `"Smith, John",Boston` gives `John"`. Why?
17. What is the difference between `cmd1; cmd2`, `cmd1 && cmd2` and `cmd1 || cmd2`?
18. You wrote `report.sh` and ran `report.sh`. You get "command not found". Give two ways to run it.
19. What do `$1`, `$#` and `$?` mean?
20. How is a job started with `&` different from one started with `nohup`?

Before next class

- Make sure your terminal works: Ubuntu in WSL on Windows, or Terminal on a Mac.
- Install `htop` and `git` if you don't have them.
- Do 15 minutes of `vimtutor`.
- Do the "Try it" exercises in the grep, sorting and scripting sections.